where does a message start?

digitalisierung und programmierung · prof. dr. nicolas meseth

1. who says now?
2. the listener
3. the end
4. the agreement

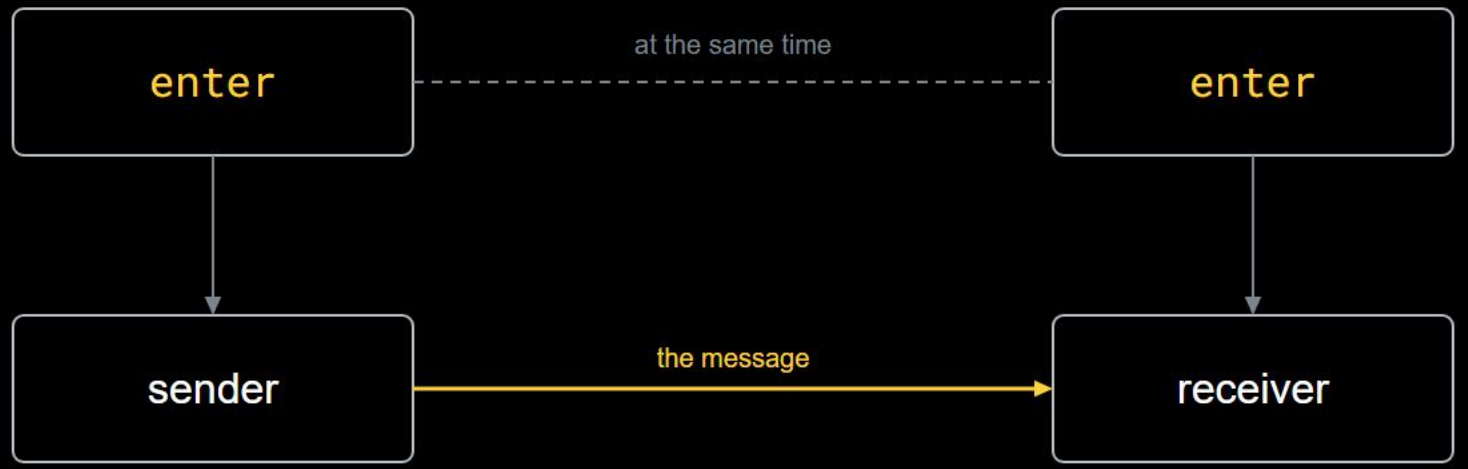
part 1

who says now?

mayday,
mayday,
mayday

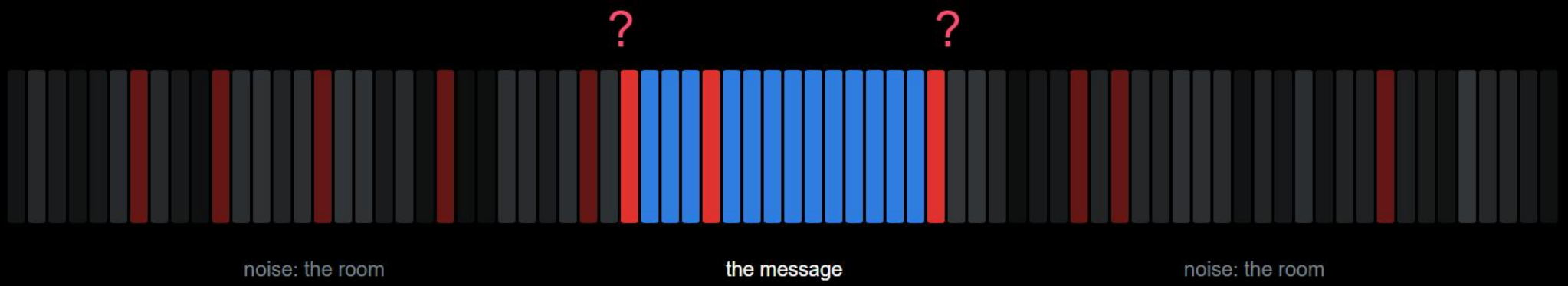
image: ai-generated (gpt-image-2)

so far, a human said "now"



someone pressed enter on both machines. that was the frame.

the receiver hears the room all the time



where does it start? where does it end? the numbers do not say.

a bit stream without a frame means nothing

bits become a message when both sides know where it starts, where it ends and what it means.

where it starts

where it starts
where it ends

where it starts
where it ends
what to do when something goes wrong

part 2

the listener

round one: i send, you write down what you see

what the room wrote down `red green green blue red red green`

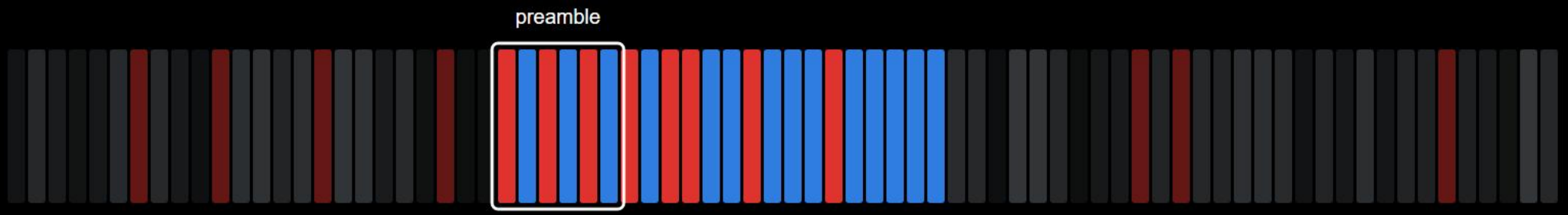
where the message was somewhere in there. nobody could say where.

no agreement, no chance. that is what your receiver sees.

what changed	every message started the same way
when the room noticed	on the second one

the third time, the room saw it coming.

you just invented the preamble



a preamble carries no information. it only says "from here on".

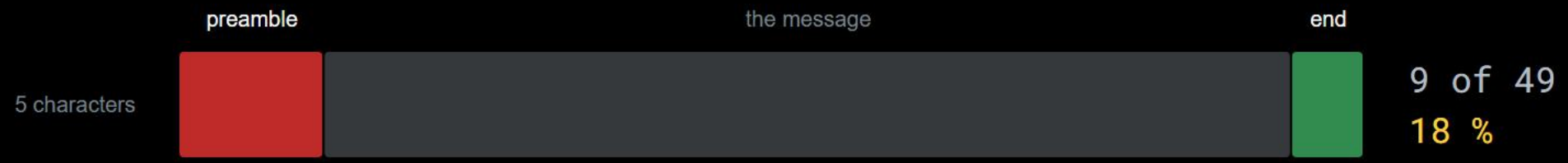
one red is not a start

start pattern: a single red

start pattern: red, blue, red, blue, red, blue

longer and more regular: rarer by chance. price: a few symbols per message.

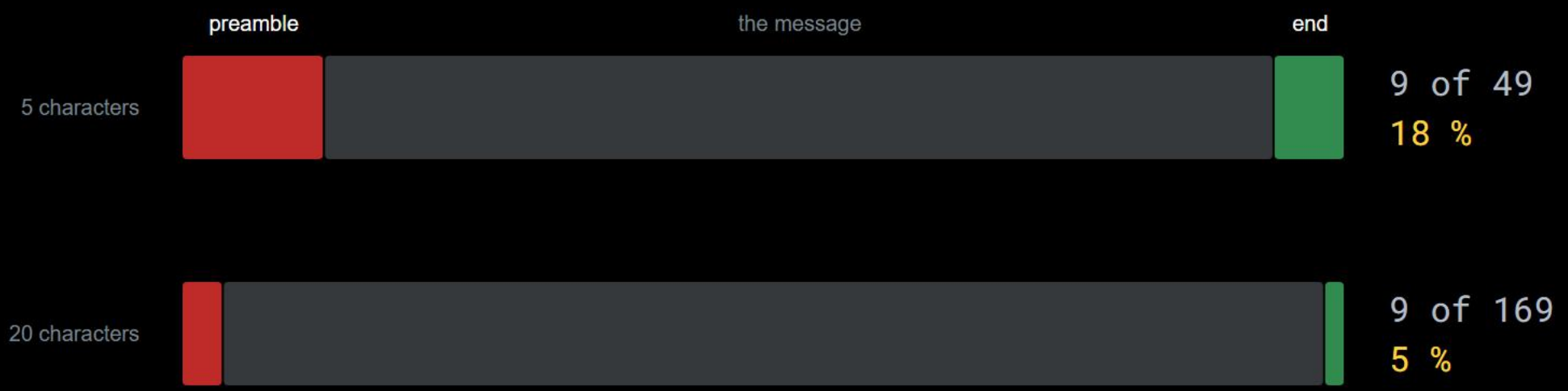
what the frame costs



6 symbols preamble, 3 symbols end marker, 8 bits per character, 2 colours

the frame costs the same per message. short messages pay most.

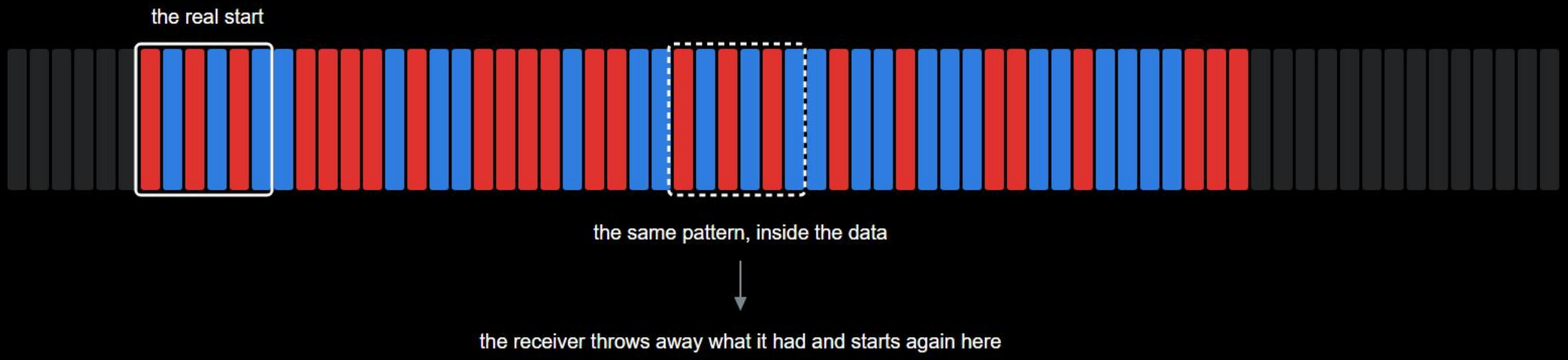
what the frame costs



6 symbols preamble, 3 symbols end marker, 8 bits per character, 2 colours

the frame costs the same per message. short messages pay most.

what if the data looks like the start



your specification must answer this. there is more than one good answer.

part 3

the end

round three: two messages, back to back

what the room proposed — say up front how long it is

and — send an agreed sign at the end

both answers are right. that is why this is a decision.

length field

"12 characters follow"

the receiver counts along

must arrive intact

neither is the right one. you pick one and write it down.

length field

"12 characters follow"

the receiver counts along

must arrive intact

end marker

"stop", a reserved symbol

any length

must never appear in the data

neither is the right one. you pick one and write it down.

the same reading means different things

noise. not the pattern. thrown away.

the reading does not decide. the state does.

the same reading means different things

a red one, but alone. the pattern needs all six.

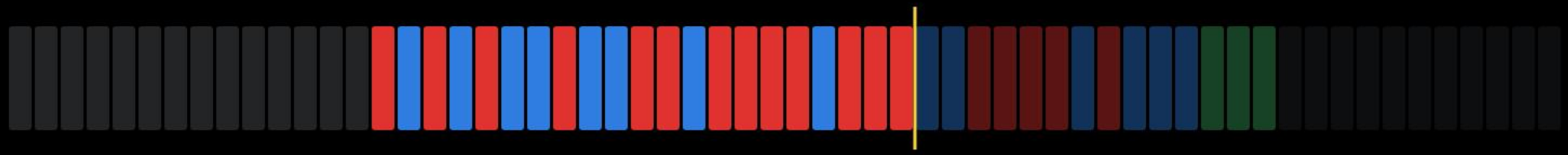
the reading does not decide. the state does.

the same reading means different things

the pattern is complete: switch to reading

the reading does not decide. the state does.

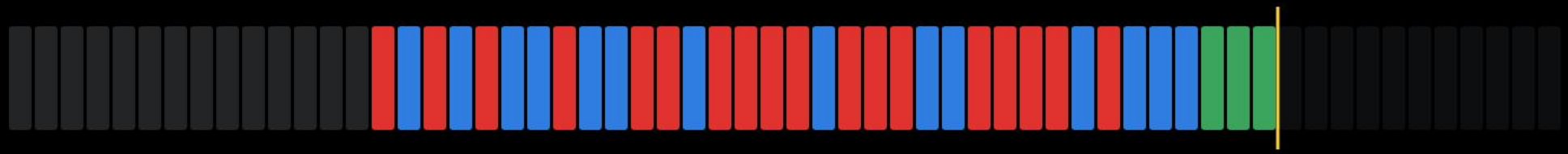
the same reading means different things



every symbol is data now, and is kept

the reading does not decide. the state does.

the same reading means different things



the end marker: hand over the message and wait again

the reading does not decide. the state does.

```
state = "waiting"
message = ""
while True:
    symbol = read_symbol()
    if state == "waiting":
        if preamble_complete(symbol):
            state = "reading"
```

forget the way back, and your receiver can read exactly one message.

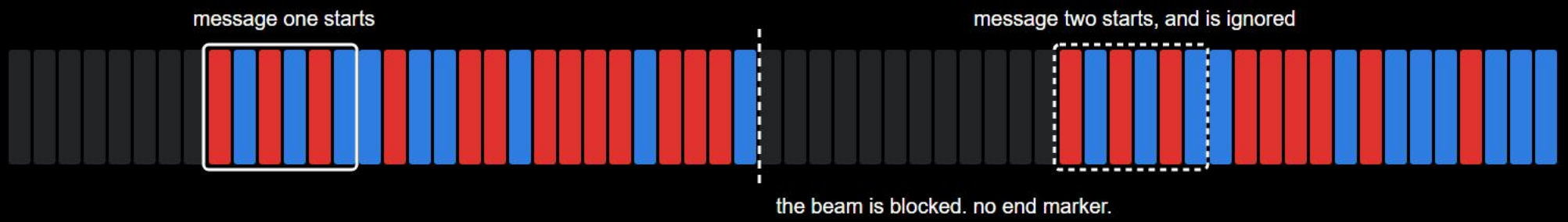
```
state = "waiting"
message = ""
while True:
    symbol = read_symbol()
    if state == "waiting":
        if preamble_complete(symbol):
            state = "reading"
    elif state == "reading":
        if symbol == END_MARK:
            print(message)


        else:
            message = message + symbol
```

forget the way back, and your receiver can read exactly one message.

```
state = "waiting"
message = ""
while True:
    symbol = read_symbol()
    if state == "waiting":
        if preamble_complete(symbol):
            state = "reading"
    elif state == "reading":
        if symbol == END_MARK:
            print(message)
            state = "waiting"
            message = ""
        else:
            message = message + symbol
```

forget the way back, and your receiver can read exactly one message.

when the end never comes



`state: reading`

it is not looking for a preamble any more,
and it never will again

it does not lose this message. it loses every message from now on.

part 4

the agreement

a protocol is an agreement someone else can implement

symbols, start, end, time, the failure case. all of it before the first bit.

the test for your specification

symbols	`0 is red, 1 is blue`
start	`red-blue, three times`
characters	`8 bits each, ascii`
end	`blue-red, three times`
how long a symbol stands	not written down anywhere
what happens if the end never comes	not written down anywhere

cross out everything you know but did not write down. is the rest enough?

this week: your own frame, written down

this week: your own frame, written down
next week: someone else builds your receiver

this week: your own frame, written down
next week: someone else builds your receiver
then: one frame for the whole class

a standard is not a technical necessity. it is an agreement.

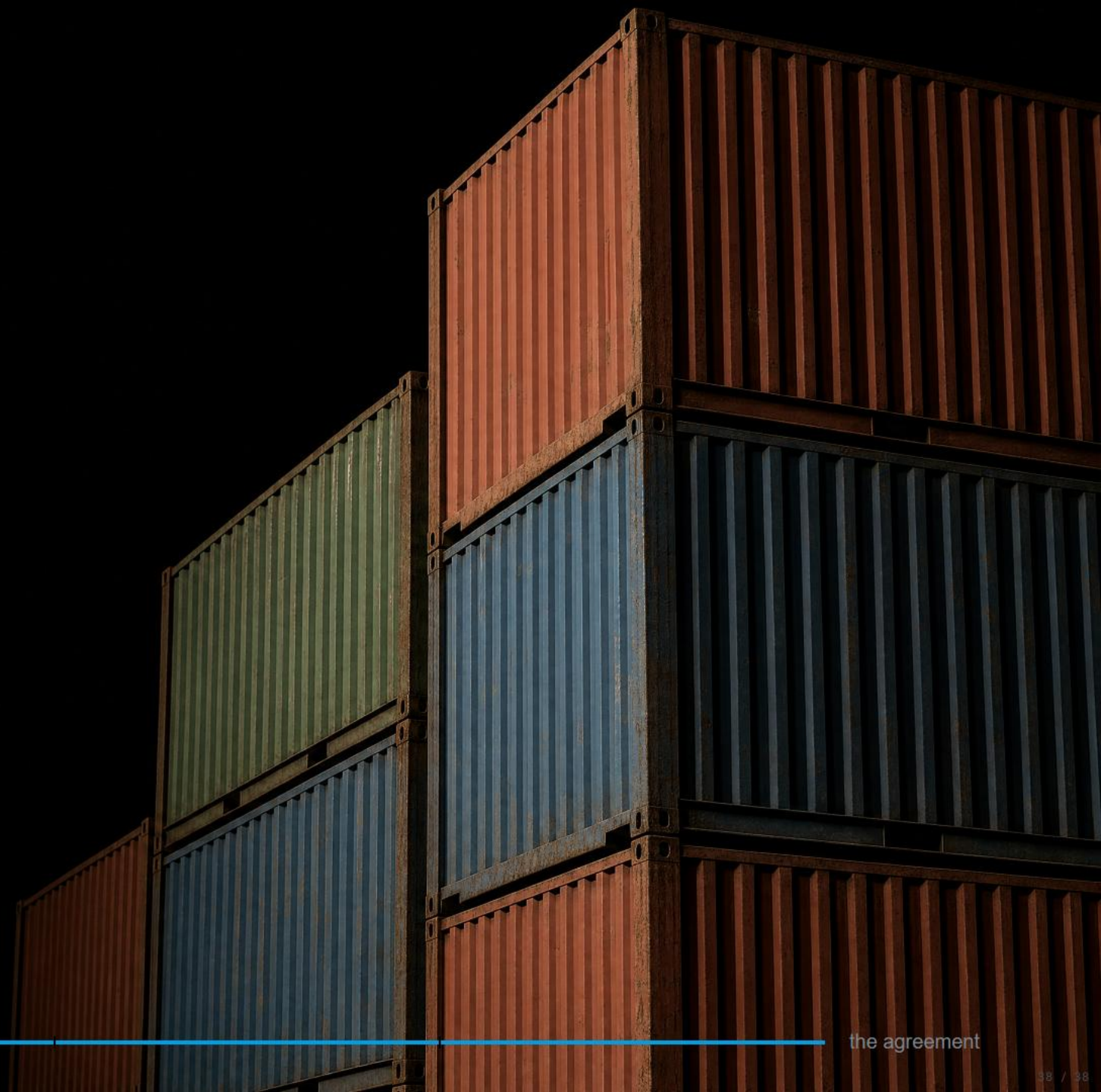


image: ai-generated (gpt-image-2)